

SYSTEM NAME: trU-Access System

GROUP NAMES:

KIMBERLY ANN ERISPE – 2024-1530-A

JAM CLAUDETTE NAPAGAO – 2024-2885-A

MAE JO SENIEL – 2024-4391-A

JOHNRIC YSULAT – 2024-0459-A

SECTION/BLOCK: BS INFO 2C

COURSE: DATABASE MANAGEMENT SYSTEMS

INSTRUCOR NAME: KENNETE ROEVEN SAYLO

DATE: JUNE 22, 2026

SYSTEM OVERVIEW

The **trU-Access**, a Campus Lost and Found Registry, is a web-based system designed to help students, staff, and administrators manage lost and found items within the campus more efficiently. Through this system, users can report items they have lost or found, browse through a list of active reports, and submit claims when they believe an item belongs to them. Admins, on the other hand, are responsible for reviewing and approving these claims, ensuring that items are returned to their rightful owners through a proper verification process. The system also keeps track of important details such as the item's category, location, date reported, and current status, making it easier to organize and search through reports.

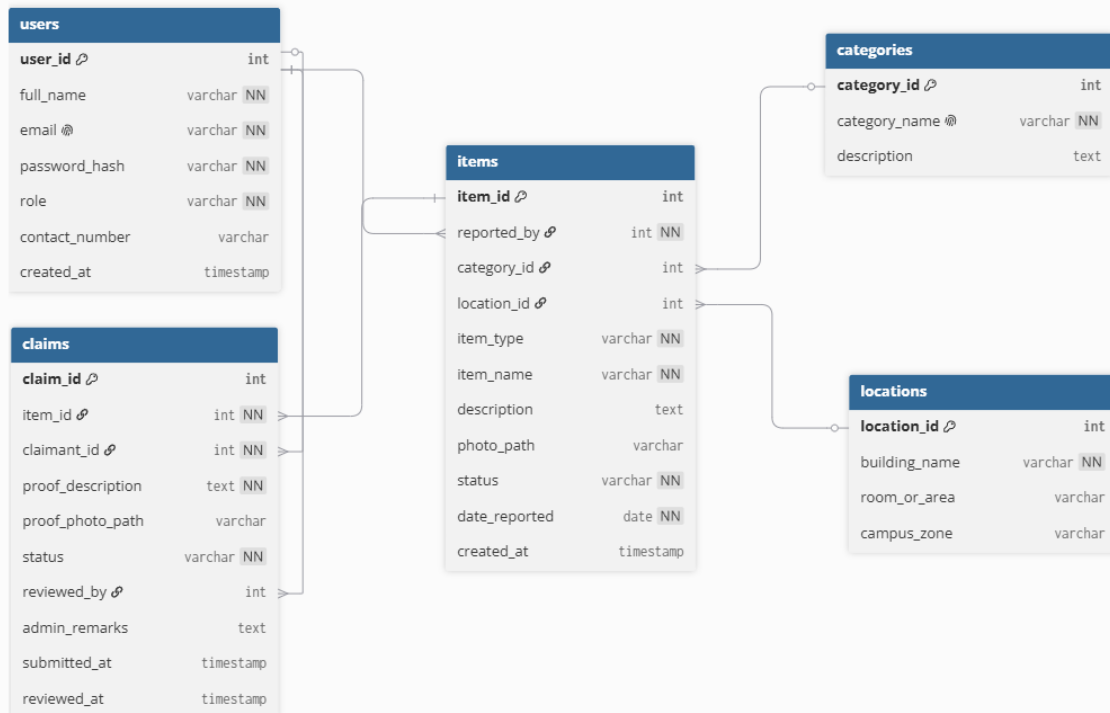
This system is intended for use by students and staff within the school campus, particularly those who often misplace personal belongings or come across items left behind in different areas like classrooms, hallways, or common areas. Before, the usual process of reporting and claiming lost items relied on physical lost and found bins, bulletin board postings, or word of mouth, which often led to items being unclaimed, misplaced again, or even wrongly given to the wrong person. Our system solves this problem by providing a centralized, organized, and more secure platform where reports and claims can be properly tracked, verified, and managed, reducing the chances of items getting lost permanently or claimed by the wrong individual.

ERD (ENTITY=RELATIONSHIP DIAGRAM)

Notation Used: Crow's Foot

- **USERS**
 - user_id (PK)
 - full_name
 - email
 - password_hash
 - role
 - contact_number
 - created_at
- **CATEGORIES**
 - category_id (PK)
 - category_name
 - description
- **LOCATIONS**
 - location_id (PK)
 - building_name
 - room_or_area
 - campus_zone
- **ITEMS**
 - item_id (PK)
 - reported_by (FK → USERS.user_id)
 - category_id (FK → CATEGORIES.category_id)
 - location_id (FK → LOCATIONS.location_id)
 - item_type
 - item_name
 - description
 - photo_path
 - status
 - date_reported
 - created_at
- **CLAIMS**
 - claim_id (PK)
 - item_id (FK → ITEMS.item_id)
 - claimant_id (FK → USERS.user_id)

- proof_description
- proof_photo_path
- status
- reviewed_by (FK → USERS.user_id)
- admin_remarks
- submitted_at
- reviewed_at



Relationships:

- USERS (1) — (M) ITEMS — *reports*
- CATEGORIES (1) — (M) ITEMS — *classifies*
- LOCATIONS (1) — (M) ITEMS — *found/lost at*
- USERS (1) — (M) CLAIMS — *submits (as claimant)*
- ITEMS (1) — (M) CLAIMS — *receives*
- USERS (1) — (M) CLAIMS — *reviews (as admin)*

Section 4 – Data Dictionary

Users

Column	Data Type	Constraints	Description
user_id	INT	PK, AUTO_INCREMENT	Unique identifier for each user.
full_name	VARCHAR	NOT NULL	Full name of the user.
email	VARCHAR	UNIQUE, NOT NULL	Email address of the user.
password_hash	VARCHAR	NOT NULL	Hashed password for login.
role	VARCHAR	NOT NULL, DEFAULT 'student', values: admin student staff	User role in the system.
contact_number	VARCHAR	NULL	Contact number of the user.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date and time the user account was created.

Categories

Column	Data Type	Constraints	Description
category_id	INT	PK, AUTO_INCREMENT	Unique identifier for each category.
category_name	VARCHAR	UNIQUE, NOT NULL	Name of the category.
description	TEXT	NULL	Additional details about the category.

Locations

Column	Data Type	Constraints	Description
location_id	INT	PK, AUTO_INCREMENT	Unique identifier for each location.
building_name	VARCHAR	NOT NULL	Name of the building.
room_or_area	VARCHAR	NULL	Room, area, or section within the building.
campus_zone	VARCHAR	NULL	Campus zone or campus section.

Items

Column	Data Type	Constraints	Description
item_id	INT	PK, AUTO_INCREMENT	Unique identifier for each item report.
reported_by	INT	FK to users.user_id, NOT NULL	User who reported the item.
category_id	INT	FK to categories.category_id, NULL	Category of the item.
location_id	INT	FK to locations.location_id, NULL	Location related to the item.
item_type	VARCHAR	NOT NULL, values: lost found	Type of item report.
item_name	VARCHAR	NOT NULL	Name of the item.
description	TEXT	NULL	Detailed description of the item.
photo_path	VARCHAR	NULL	Path or URL to the item photo.
status	VARCHAR	NOT NULL, DEFAULT 'active', values: active claimed archived	Current status of the item.
date_reported	DATE	NOT NULL	Date the item was reported.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date and time the item record was created.

Claims

Column	Data Type	Constraints	Description
claim_id	INT	PK, AUTO_INCREMENT	Unique identifier for each claim.
item_id	INT	FK to items.item_id, NOT NULL	Item being claimed.
claimant_id	INT	FK to users.user_id, NOT NULL	User who submitted the claim.
proof_description	TEXT	NOT NULL	Description of proof of ownership.
proof_photo_path	VARCHAR	NULL	File path or photo used as proof.
status	VARCHAR	NOT NULL, DEFAULT 'pending', values: pending approved rejected	Claim status.
reviewed_by	INT	FK to users.user_id, NULL	Admin or staff who reviewed the claim.

admin_remarks	TEXT	NULL	Reviewer remarks or notes.
submitted_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date and time the claim was submitted.
reviewed_at	TIMESTAMP	NULL	Date and time the claim was reviewed.

Section 5 — SQL Script Summary

Stored Procedure

Procedure Name: ApproveClaim

What it does: Updates a claim's status to "approved," logs which admin reviewed it and when, then automatically changes the related item's status to "claimed."

DELIMITER //

```
CREATE PROCEDURE ApproveClaim (  
    IN p_claim_id INT,  
    IN p_admin_id INT,  
    IN p_remarks TEXT  
)  
BEGIN  
    UPDATE claims  
    SET status = 'approved',  
        reviewed_by = p_admin_id,  
        admin_remarks = p_remarks,  
        reviewed_at = CURRENT_TIMESTAMP  
    WHERE claim_id = p_claim_id;  
  
    UPDATE items  
    SET status = 'claimed'  
    WHERE item_id = (  
        SELECT item_id FROM claims WHERE claim_id = p_claim_id  
    );  
END //
```

DELIMITER ;

How to use it:

1. Run the CREATE PROCEDURE script once in your database (e.g., in MySQL Workbench or phpMyAdmin) — this only needs to be done one time to register the procedure.
2. Whenever an admin approves a claim in the system (e.g., clicks "Approve" on the web page), the backend code calls this procedure instead of writing two separate UPDATE statements.
3. To run it manually for testing:
4. `CALL ApproveClaim(15, 2, 'Verified school ID matches claimant description.');`
 - **15 = the claim_id being approved**
 - **2 = the user_id of the admin approving it**
 - **'Verified school ID...' = the remarks the admin typed in**
5. After running, check both tables to confirm the update worked:

```
SELECT * FROM claims WHERE claim_id = 15;  
SELECT * FROM items WHERE item_id = (SELECT item_id FROM claims  
WHERE claim_id = 15);
```

View

View Name: ActiveLostItemsView

What it does: Combines items, categories, locations, and users into one ready-made table showing only items that are still active (not yet claimed or archived).

```
CREATE VIEW ActiveLostItemsView AS
```

```
SELECT
```

```
  i.item_id,
```

```
  i.item_name,
```

```
  i.item_type,
```

```
  i.status,
```

```
  i.date_reported,
```

```
  c.category_name,
```

```
  l.building_name,
```

```
l.room_or_area,  
u.full_name AS reported_by_name  
FROM items i  
JOIN categories c ON i.category_id = c.category_id  
JOIN locations l ON i.location_id = l.location_id  
JOIN users u ON i.reported_by = u.user_id  
WHERE i.status = 'active';
```

How to use it:

1. Run the CREATE VIEW script once — this saves the query permanently as a virtual table named ActiveLostItemsView.
2. From then on, instead of writing the full JOIN every time the dashboard or "Browse Items" page needs to display active items, the application just runs:

```
SELECT * FROM ActiveLostItemsView;
```

3. This is useful for the Dashboard/List page in your system — it's the exact query the page would call to populate the table of active lost/found items.
4. You can also filter it further, e.g., to show only "found" items:

```
SELECT * FROM ActiveLostItemsView WHERE item_type = 'found';
```

JOIN Query

What it does: Pulls together claim details with the item info and the claimant's contact info — useful when an admin opens a specific item to review who is claiming it.

```
SELECT  
cl.claim_id,  
i.item_name,  
u.full_name AS claimant_name,  
u.contact_number,  
cl.status,  
cl.submitted_at
```

```
FROM claims cl  
JOIN items i ON cl.item_id = i.item_id  
JOIN users u ON cl.claimant_id = u.user_id  
WHERE i.item_id = 7;
```

How to use it:

1. This query is run when an admin clicks into a specific item's "View Claims" page.
2. Replace 7 with whatever item_id the admin clicked on (your backend code would pass this value dynamically).
3. The result lets the admin see, at a glance, who has claimed the item, how to contact them, and whether the claim is still pending.

Aggregate Query

What it does: Counts how many items fall under each category, useful for generating reports or charts.

```
SELECT  
    c.category_name,  
    COUNT(i.item_id) AS total_items_reported  
FROM items i  
JOIN categories c ON i.category_id = c.category_id  
GROUP BY c.category_name  
ORDER BY total_items_reported DESC;
```

How to use it:

1. This query is typically used on an admin reports/statistics page, e.g., a bar chart showing "Electronics: 23 items, IDs: 15 items," etc.
2. Run it directly in your database tool to test, or hook it up to your backend so it returns JSON data that a chart library (like Chart.js) can render.
3. You can adjust the ORDER BY and add a LIMIT 5 if you only want the top 5 most-reported categories.

Subquery Example

What it does: Finds all users who have submitted at least one claim, using a subquery to check against the claims table.

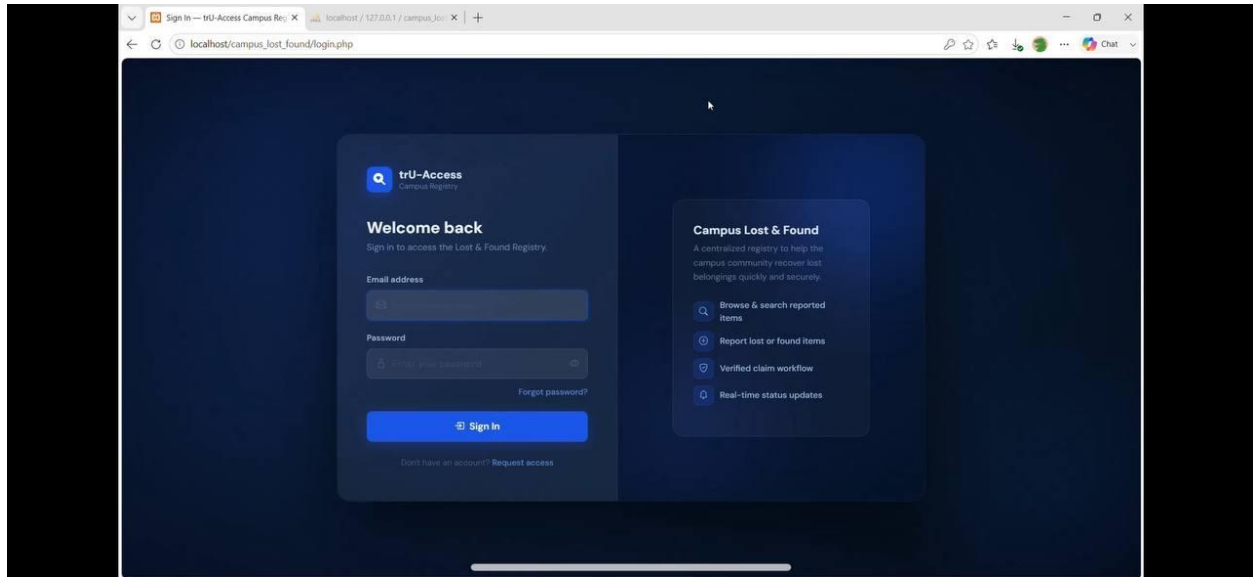
```
SELECT
    user_id,
    full_name,
    email
FROM users
WHERE user_id IN (
    SELECT DISTINCT claimant_id FROM claims
);
```

How to use it:

1. This is useful for an admin user-management page where you want to filter and see only users who have actually been active in the claims process (versus users who registered but never claimed anything).
2. The inner query (SELECT DISTINCT claimant_id FROM claims) runs first and produces a list of all unique claimant IDs.
3. The outer query then matches those IDs against the users table to pull their names and emails.
4. You could reuse this same pattern to find the opposite — users who have never submitted a claim — by changing IN to NOT IN.

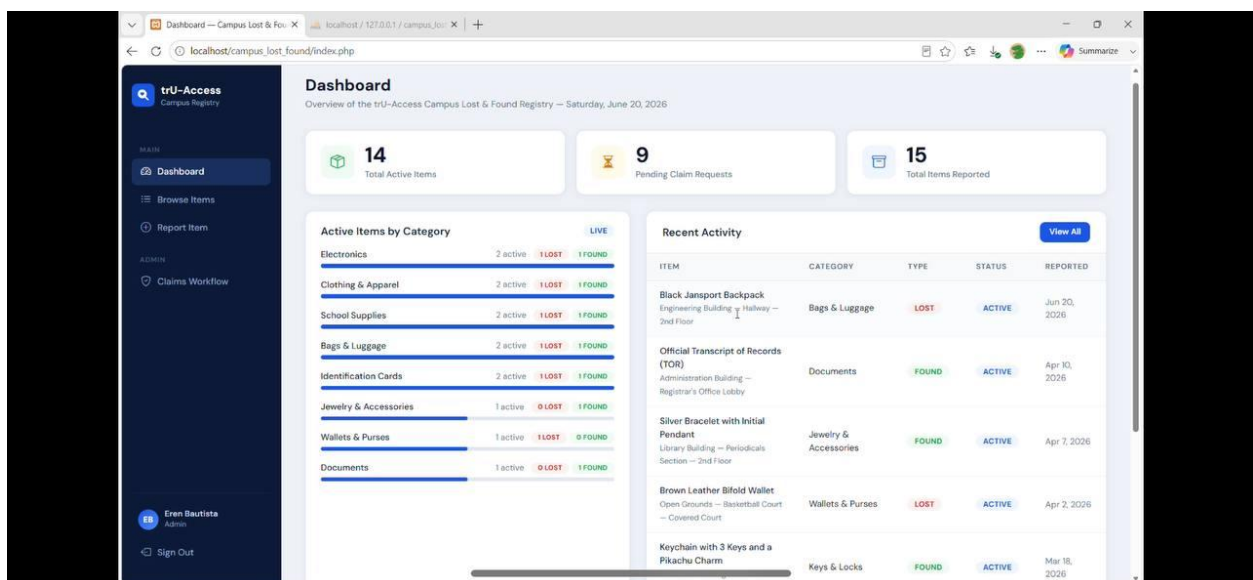
Section 6 – System Screenshots

Login Page - This is the entry point of the system where users (students, staff, or admins) input their email and password to access their account. It checks if the credentials match what's stored in the database and directs the user to their respective dashboard based on their role (admin or regular user).

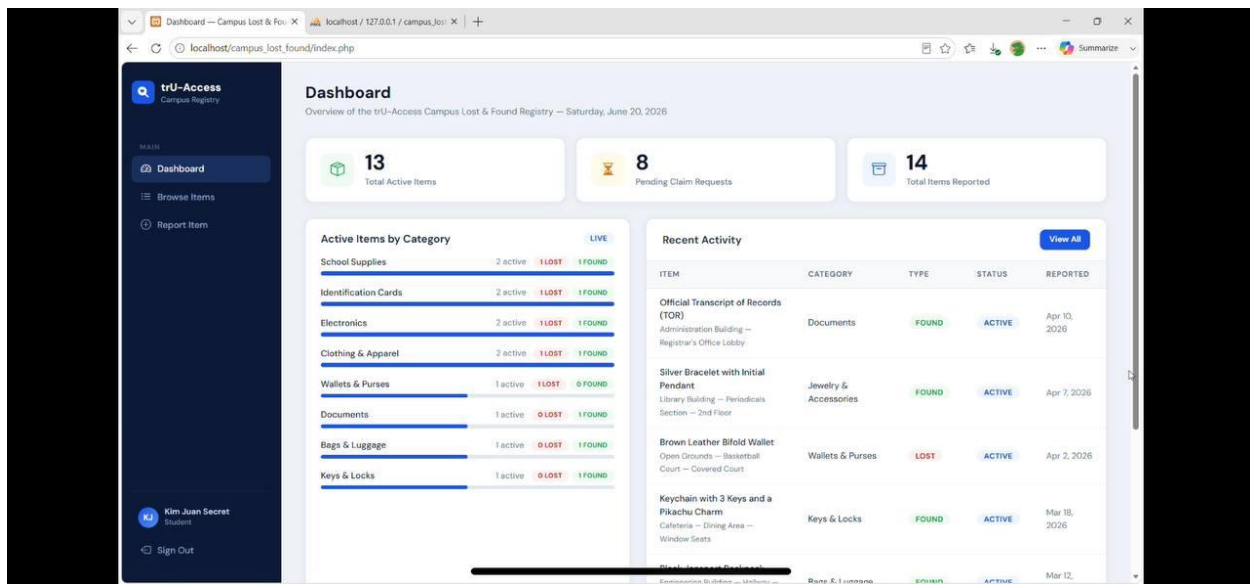


Dashboard Page - This page serves as the main hub once a user logs in. It gives a quick overview of recent activity, such as newly reported items, pending claims, or notifications, depending on whether the user is an admin or a regular student/staff. It acts as the starting point for navigating to the other pages of the system.

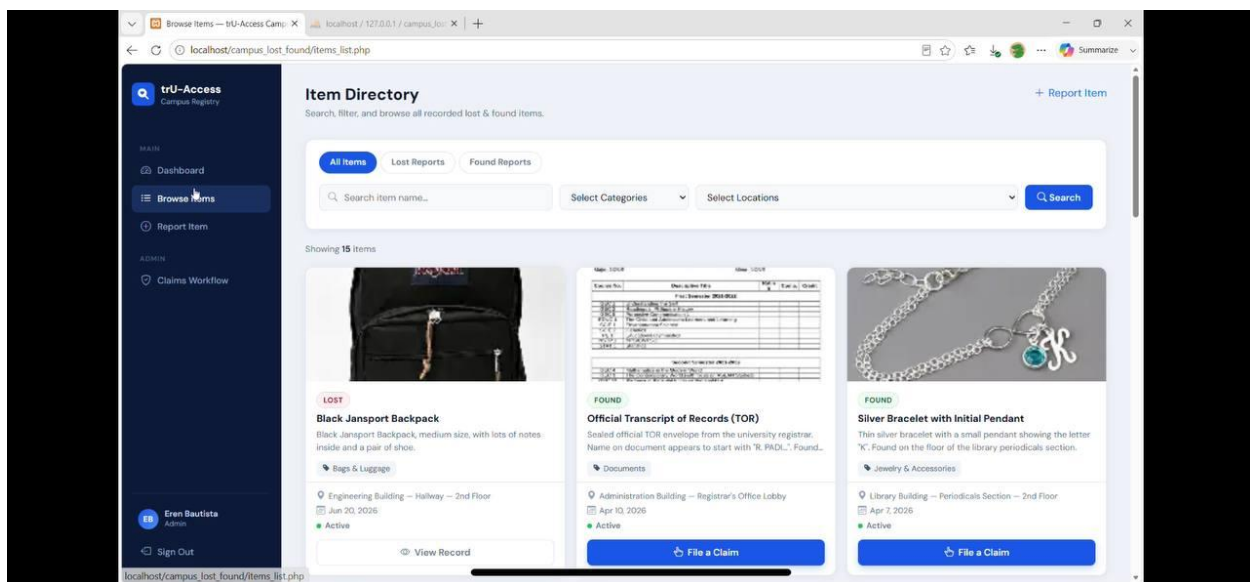
Admin's Dashboard



Student's Dashboard



Browse Item Page - This page displays a list of all active lost and found items reported in the system. Users can search or filter items by category, location, or item type (lost/found) to easily find something that matches what they're looking for. Each item usually shows basic details like the item name, photo, and where/when it was found or lost.



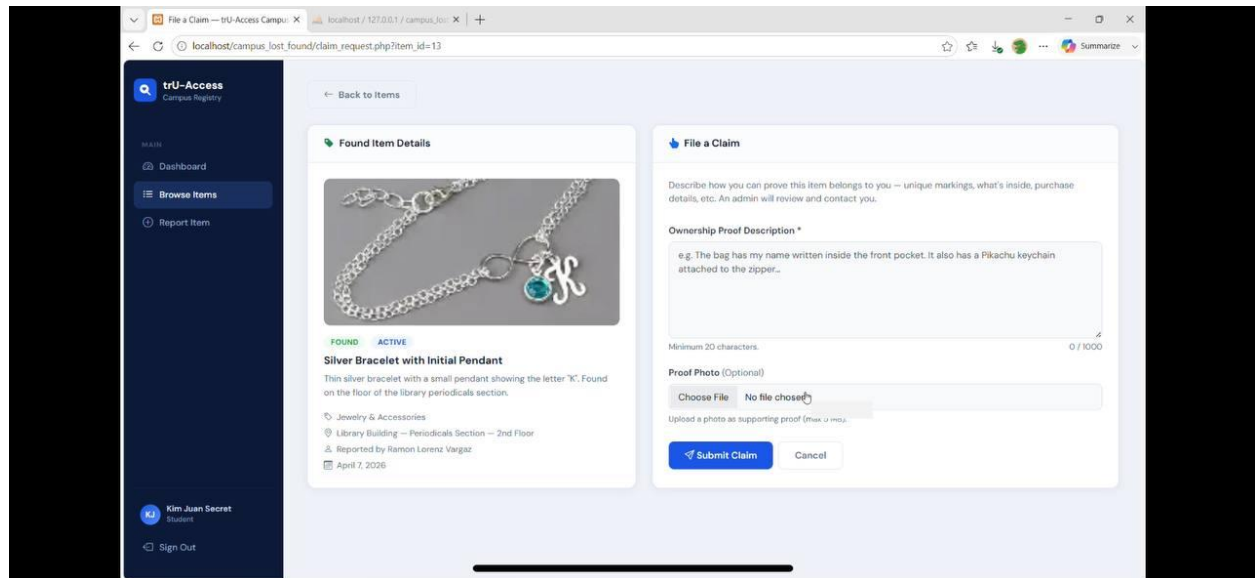
Report Item Page - This is where users submit a new report for an item they've lost or found. They fill out details such as the item name, description, category, location, and upload a photo if available. Once submitted, the report becomes visible on the Browse Item page for others to see.

The screenshot shows the 'report an item' page in a web browser. The page title is 'report an item' with a subtitle 'Log a lost or found item with details and an optional photo.' The left sidebar contains the 'trU-Access Campus Registry' logo and navigation links: 'Dashboard', 'Browse Items', 'Report Item' (highlighted), and 'Claims Workflow'. The main content area is a form titled 'Item Details' with the following fields: 'Item Type' (radio buttons for 'Lost Item' and 'Found Item'), 'Item Title' (text input with example 'e.g. Black Jansport Backpack, Blue Umbrella...'), 'Description' (text area with placeholder 'Describe the item - colour, brand, markings, contents, etc.'), 'Category' (dropdown menu), 'Location' (dropdown menu), 'Date Reported' (date input showing '06/20/2026'), and 'Photo (Optional)' (image upload area). A 'Sign Out' button is visible in the sidebar.

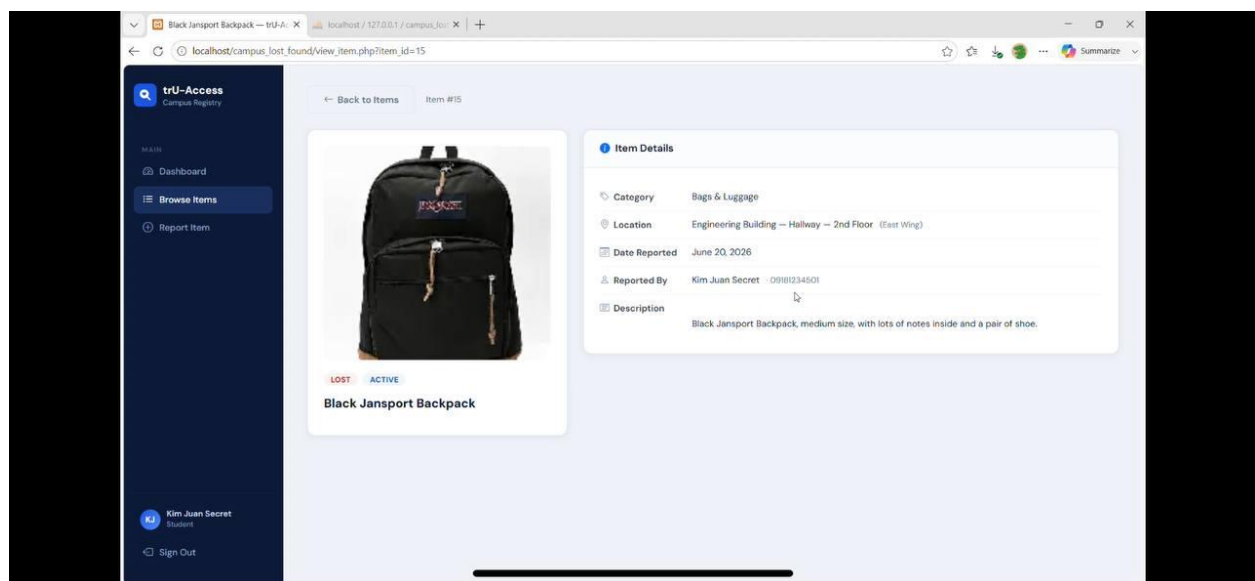
Claims Workflow (Pending Claims) - This page is mainly used by admins to view and manage all claims that are still awaiting review. It lists each claim's status, the claimant's information, and the item being claimed, allowing the admin to approve or reject claims after verifying the proof submitted.

The screenshot shows the 'Claims Workflow' page in a web browser. The page title is 'Claims Workflow' with a subtitle 'Review and process pending ownership claims.' The left sidebar is identical to the previous page, with 'Claims Workflow' highlighted. The main content area shows a list of 'Pending Claims' with a '9 PENDING' badge. Each claim entry includes: 'FOUND' and 'ACTIVE' status indicators, 'Claim #', item name, location, date, claimant name and email, and an 'OWNERSHIP PROOF' section. The first claim is for a 'Silver Bracelet with Initial Pendant' by Eren Bautista, dated April 7, 2026. The second is for an 'Official Transcript of Records (TOR)' by Andres Ramel Navarro Jr., dated April 10, 2026. The third is for a 'Black Jansport Backpack' by Maria Jo Seniel, dated May 12, 2026. Each claim has 'Approve' and 'Reject' buttons. A 'Sign Out' button is visible in the sidebar.

File a Claim - This page allows a user to submit a claim for an item they believe belongs to them. They need to provide proof, such as a description or photo, to support their claim, which will then be reviewed by an admin before it gets approved or rejected.



View Claim - This page lets users check the status of the claims they've submitted, whether it's still pending, approved, or rejected. Admins can also use a similar page to view full details of a specific claim, including the proof submitted and any remarks made during the review process.



Section 7 – Individual Contribution Summary

KIMBERLY ANN ERISPE

Working on this project, I was the one who handled the topic proposal at the very start. I had to think carefully about what kind of system would actually be useful for our campus, and that's how we landed on the Lost and Found Registry. It took a few revisions before our proposal got approved, but I learned that planning ahead really saves time later on.

After the proposal, I moved on to creating the database for our system. I had to study how the tables should connect to each other, like how items relate to users and categories. It was a bit overwhelming at first since I'm not that confident with SQL, but I just kept practicing until the relationships made sense.

I also spent a lot of time searching for the codes we needed for different parts of the system. Since we were using a framework that none of us were fully familiar with, I had to look through documentation and sample projects just to find the right snippets we could use as reference. It was tiring but I'm thankful for tutorials and forums online.

Another task I handled was the screen recording of our system. I made sure to capture every major feature so our group would have enough footage for the video presentation later. I had to redo some recordings a few times because of small mistakes, but it taught me to be more patient and detail-oriented.

Lastly, I worked on the documentation part of our project. I organized our ERD, data dictionary, and other written requirements into one file. Writing documentation made me realize how important it is to explain technical things in a simple way, since not everyone reading it will understand code.

Overall, this project taught me that building a system is not just about writing code, it's also about planning, researching, and communicating clearly. I'm proud of what our group accomplished even with the struggles we had along the way. This experience definitely helped me grow more confident in handling bigger tasks.

MAE JO SENIEL

My main role in this project was checking and debugging the codes that our groupmates encoded. Since I was the one reviewing the code closely, I had to really understand what each part was doing so I could spot errors properly. It was challenging at times because some bugs were hard to trace, but it helped me improve my problem-solving skills.

I also did a lot of double-checking on my own laptop to make sure the system ran smoothly outside of just one device. There were times the system worked fine on one laptop but had issues on mine, so I had to figure out what caused the difference. This made me realize how important it is to test a system on different setups before finalizing it.

Whenever we encountered errors, I searched for possible solutions online, whether through forums, documentation, or just trial and error. Some errors took hours to fix, especially when the error message wasn't very clear. I learned to be more patient and to read error logs carefully instead of guessing right away.

Aside from debugging, I was also in charge of editing our video presentation. I combined our screen recordings and added the necessary captions and transitions to make it look presentable. Editing videos was new to me, so I had to learn some basic editing skills just for this project.

Looking back, this project taught me that debugging is honestly one of the most time-consuming parts of system development. It also showed me the value of teamwork, since fixing errors was easier when we discussed solutions together. I'm grateful for this experience because it pushed me to learn things outside my comfort zone.

JAM CLAUDETTE NAPAGAO

For this project, my laptop was the main device we used in building our entire system. This meant I had a big responsibility in keeping all our files organized and accessible to the group. I made sure everything was properly saved and arranged so we wouldn't lose any progress along the way.

My main task was encoding all the codes for our system. I had to type out the structure, the queries, and the logic based on what we planned for our database and features. It took a lot of focus since one small typo could cause the whole system to break.

I was also in charge of arranging the files in our project folder, making sure everything was placed in the correct directories. This helped our group avoid confusion, especially when we had multiple files for different parts of the system. I learned that proper file organization actually saves a lot of time when something needs to be fixed or reviewed later.

One specific task I handled was manually adding the hashed passwords for our user accounts. I had to research how password hashing works and make sure I implemented it correctly so our system would be more secure. This was a good learning experience because I got to understand why we shouldn't store plain passwords in a database.

Aside from coding, I also contributed to parts of our documentation, helping explain some of the technical sections in simpler terms. This whole project showed me how much patience and attention to detail is needed in actually building a working system, not just learning the theory behind it.

JOHNRIC YSULAT

My main contribution to this project was writing the script for our video presentation. I had to think about how to explain our system in a clear and organized way so that anyone watching, even those without a technical background, could understand what we built. Writing the script took several drafts before it felt natural and not too robotic.

After finishing the script, I also handled the recording and acted as the videographer for our presentation. I made sure the audio and visuals were clear, and I had to record some parts multiple times to get a clean take. This task taught me that presentation matters just as much as the system itself, since it's how we communicate our hard work to others.

I also contributed to some of the expenses needed for our project, whether it was for printing documents or other small project needs. Even though it wasn't a technical task, I learned that group projects also involve managing resources and being responsible with shared costs.

Lastly, I helped debug some of the errors we encountered in the code. While I wasn't the main one assigned to checking the system, I still tried to help out whenever our groupmates needed an extra hand, especially when multiple errors needed fixing at the same time. This experience reminded me that teamwork means stepping up wherever you're needed, not just sticking to one role.

Overall, this project helped me realize that a system isn't just about the code, it's also about how you present and explain it to others. I'm thankful for the chance to contribute in different ways, even outside of just programming.